

TALLER 1: FLUTTER + WIDGETS + GIT FLOW

INFORME DE EVIDENCIAS TÉCNICAS, ARQUITECTURA REACTIVA Y CONTROL DE VERSIONES

Institución: Unidad Central del Valle del Cauca (UCEVA)	Asignatura: Electiva Profesional 1 Móviles
Estudiante: Samuel Alejandro Rincon Serna	Código: 230231045
Repositorio GitHub: https://github.com/Sarincon508/flowbiz	Rama Taller: feature/taller1 (Integrada a dev y main)
Entorno de Ejecución: Android Emulator (Pixel 7 • API 34)	Fecha de Entrega: 23 de Septiembre de 2026

1. Objetivo del Taller

Construir una pantalla interactiva y profesional en Flutter implementando **StatefulWidget** y evidenciando la mutación reactiva de la interfaz mediante **setState()**. Asimismo, integrar buenas prácticas de desarrollo móvil incluyendo diseño jerárquico (Column, Row, Padding, SizedBox), renderizado híbrido de imágenes locales (Image.asset()) y remotas (Image.network()), retroalimentación inmediata al usuario con **SnackBar** flotante, e incorporación de widgets adicionales. Finalmente, aplicar una disciplina estricta de control de versiones basada en **Git Flow** sobre el repositorio oficial del curso en GitHub.

2. Fundamento de StatefulWidget y setState() en la Solución

En Flutter, los widgets son inmutables por definición. Para gestionar componentes visuales cuyos datos cambian en respuesta a eventos del usuario o procesos asíncronos, se utiliza **StatefulWidget**, el cual delega su ciclo de vida y variables a una clase complementaria **State<HomePage>**.

- **Variable de Estado:** Se definió String _tituloAppBar = 'Hola, Flutter'; y int _contadorCambios = 0;.
- **Método setState():** Al presionar el botón *ElevatedButton*, se ejecuta _alternarTitulo(), envolviendo la actualización de las variables dentro de setState() { ... }. Esto notifica formalmente al framework de Flutter que el estado interno ha mutado, desencadenando la ejecución eficiente de build() para reconstruir únicamente los subárboles de widgets dependientes (el título del AppBar, el texto sincronizado en el Stack y el badge).

3. Evidencias de Ejecución en Emulador Android (API 34)

A continuación se documentan las capturas de pantalla reales tomadas directamente desde el emulador activo (emulator-5554) utilizando la herramienta adb screencap:

Figura 1: Estado Inicial de la App	Figura 2: Botón Presionado + setState() + SnackBar
------------------------------------	--

 • Título AppBar: Hola, Flutter • Nombre de estudiante y código centrados. • Galería en Row: Image.network() (búho CDN) e Image.asset() (recurso local). • Botón listo para interacción.	 • Título AppBar mutado: ¡Título cambiado! • Contador de interacciones incrementado a 3. • SnackBar flotante activo: 'Título actualizado (Estado: ¡Título cambiado!)'. • Stack sincronizado con el nuevo título.
--	--

4. Evidencias y Análisis de Widgets Adicionales Implementados

El taller exigía incorporar al menos dos widgets adicionales de una lista selecta. En esta solución se diseñaron e implementaron con excelencia visual cuatro componentes fundamentales:

Figura 3: Widgets Adicionales (Stack, ListView, Botones)	Detalle Técnico de Implementación
 The screenshot displays a mobile application interface. At the top, a dark blue header bar contains the text "¡Título cambiado!" and a close icon. Below the header, there are two image placeholders: "Image.network()" and "Image.asset()". A prominent blue button labeled "Restaurar a 'Hola, Flutter'" is visible. The main content area is divided into two sections. The first section, titled "Widget Adicional: Stack (Superposición de Texto y Gráficos)", shows a dark blue container with a "MODULO REACTIVO ACTIVO" label and a "Título actual: '¡Título cambiado!'" text. The second section, titled "Widget Adicional: ListView (Módulos y Competencias del Taller)", contains a list of four items: "1. StatefulWidget & setState()", "2. Renderizado de Imágenes", "3. Flujo Git Flow Profesional", and "4. Layouts y Composición Limpia". At the bottom, there are two buttons: "Reiniciar Estado" and "Ir a FlowBiz App".	1. Container Estilizado: Encabeza la interfaz con gradiente azul institucional UCEVA Navy (#1B365D a #0284C7), bordes redondeados (16px), sombras gaussianas de elevación, divisor interior y presentación centrada de los datos del estudiante. 2. Stack (Superposición Gráfica Reactiva): Superpone sobre un fondo de alta densidad con gradiente oscuro una marca de agua del icono oficial de Flutter Dash, una etiqueta de estatus en esquina superior izquierda, y texto dinámico en la parte inferior que muta de forma inmediata al cambiar el título con <code>setState()</code>. 3. ListView con ListTile: Lista estructurada de 4 tarjetas informativas con scroll controlado, presentando las competencias del taller, cada una con icono distintivo en contenedor cromático, título jerárquico y descripción detallada. 4. OutlinedButton & ElevatedButton: Botones de acción secundaria para restablecer el estado inicial a valores por defecto y navegación directa al flujo completo del aplicativo FlowBiz.

5. Evidencia de Flujo de Ramas Git Flow

El ciclo de vida del código fuente se gestionó íntegramente siguiendo el modelo profesional de Git Flow:

- Rama main:** Rama estable de producción.
- Rama dev:** Creada a partir de main para consolidar las fases de desarrollo.
- Rama feature/taller1:** Rama de trabajo aislada nacida desde dev, donde se implementaron los commits atómicos del Taller 1 (StatefulWidget, imágenes, widgets adicionales, pruebas y documentación).
- Pull Request e Integración:** Simulación de Pull Request con fusión de tipo non-fast-forward (--no-ff) de feature/taller1 hacia dev, y posterior integración final de dev hacia main.
- Verificación Remota:** Ambas ramas dev y feature/taller1 se encuentran publicadas en GitHub. La rama main cuenta con reglas activas de Branch Protection que restringen pushes directos y exigen Pull Request.

6. Extracto de Código Fuente: Gestión del Estado con setState()

A continuación se presenta el fragmento de código de lib/screens/taller1_home_page.dart donde se evidencia la manipulación del estado y la activación del Snackbar:

```
class _HomePageState extends State<HomePage> { String _tituloAppBar = 'Hola, Flutter'; int _contadorCambios = 0; void
_alternarTitulo() { setState(() { if (_tituloAppBar == 'Hola, Flutter') { _tituloAppBar = '¡Titulo cambiado!'; } else {
_tituloAppBar = 'Hola, Flutter'; } _contadorCambios++; }); ScaffoldMessenger.of(context).hideCurrentSnackBar();
ScaffoldMessenger.of(context).showSnackBar( SnackBar( content: Text('Titulo actualizado (Estado: $_tituloAppBar)'),
backgroundColor: const Color(0xFF1B365D), duration: const Duration(seconds: 2), behavior: SnackBarBehavior.floating, ), );
} ... }
```

7. Lista de Comprobación y Verificación de Requisitos

Verificación formal de cada uno de los ítems requeridos en la guía del taller:

Ítem de la Guía Oficial	Detalle de Cumplimiento Técnico	Estado
■ Repositorio público creado y accesible en GitHub	https://github.com/Sarincon508/flowbiz	CUMPLIDO
■ Ramas main y dev configuradas según Git Flow	main protegida y dev rama base	CUMPLIDO
■ feature/taller1 creada desde dev	Desarrollo completo en rama de características	CUMPLIDO
■ StatefulWidget implementado	HomePage hereda de StatefulWidget con clase State	CUMPLIDO
■ setState() implementado y funcional	Alternancia de títulos y contador reactivo	CUMPLIDO
■ Image.network() implementado	Carga desde CDN oficial de Flutter con fallbacks	CUMPLIDO
■ Image.asset() implementado	Carga local desde assets/images/flutter_taller.png	CUMPLIDO
■ Mínimo 2 widgets adicionales implementados	4 widgets: Container, Stack, ListView, OutlinedButton	CUMPLIDO
■ Snackbar funcionando	Mensaje flotante: 'Título actualizado' al presionar botón	CUMPLIDO
■ README.md completado con instrucciones y capturas	Documentación exhaustiva en repo y workspace	CUMPLIDO
■ PR feature/taller1 -> dev realizado	Fusión no fast-forward con mensaje de Pull Request	CUMPLIDO
■ Integración dev -> main realizada	Release consolidado en rama de producción	CUMPLIDO
■ PDF de evidencias preparado para entrega en Moodle	Documento formal institucional UCEVA generado	CUMPLIDO